

Operational Manual: T-Matrix Extraction from CST Studio Suite Frequency-Domain Simulations

`cst_tmatrix` package documentation

August 10, 2026

Abstract

This manual documents the theory, software architecture, operating procedure, and validation of the `cst_tmatrix` package, which computes transition matrices (T-matrices) of arbitrary, spatially localized scatterers from full-wave simulations performed with the frequency-domain (FDFD) solver of CST Studio Suite. The incident and scattered multipole amplitudes are both obtained from the simulated total field on a spherical surface, which makes the extraction independent of the amplitude and phase conventions of the excitation. Results are stored in the community `tmat.h5` data format [12] used by the multiple-scattering code *treams* [13]. The method follows the formulation of the Neural Analytic Modal Method (NAMM) document [14], with the conventions closed and verified as described here.

Contents

1	Introduction	2
2	Theoretical formulation	3
2.1	Conventions and vector spherical waves	3
2.2	Plane-wave expansion	3
2.3	Multipole truncation	4
2.4	Self-calibrating field separation	4
2.5	Monitor placement	5
2.6	Least-squares extraction and illumination set	6
2.7	Symmetry-reduced illumination	6
2.8	Surface quadrature	8
2.9	Diagnostics	8
2.10	Analytic references: homogeneous and coated spheres	9
2.11	Conversion to the <code>tmat.h5</code> convention	9
3	Simulation setup in CST Studio Suite	9
4	Software	11
4.1	Modules	11
4.2	Installation and configuration	11
4.3	Operating procedure	12
4.4	Automatic checks and unattended operation	12

5	Usage	13
5.1	Scripted geometries	13
5.2	Template projects	14
5.3	Extraction from a configuration file	14
5.4	Extraction options	15
5.5	Output files	15
6	Validation	15
6.1	Offline tests	15
6.2	Comparison with full-wave reference solutions	16
6.3	The absolute scale of the extraction error	16
6.4	Benchmark against an independent solver: TiO ₂ cylinder	17
6.5	Recommended validation sequence for new object classes	17
7	Sources of error and how they manifest	17
8	Planned extensions	18
8.1	Metasurface post-processing: implemented stages	18
8.2	Planned: periodic lattice sum and substrate coupling	19
A	CST-specific behaviors and workarounds (bug report)	19

1 Introduction

The transition matrix (T-matrix) of a localized object relates the coefficients of an incident electromagnetic field, expanded in regular vector spherical waves, to the coefficients of the scattered field, expanded in outgoing vector spherical waves [1, 3]. It encodes the complete linear electromagnetic response of the object at a given frequency and is the natural building block for multiple-scattering computations of clusters, metasurfaces, and disordered media through the Foldy–Lax equations and translation-addition theorems [4, 5].

For objects without analytic solutions the T-matrix must be computed numerically. The approach implemented here illuminates the object with a set of plane waves from distributed directions, solves each scattering problem with the frequency-domain solver of CST Studio Suite, records the complex electric and magnetic fields on a spherical surface enclosing the object, and determines the T-matrix from the least-squares relation between the incident and scattered expansion coefficients [6]. Two properties distinguish the present implementation:

1. *Self-calibrating field separation.* The incident and scattered coefficients are both extracted from the same recorded total field (Sec. 2.4); no assumption about the source amplitude or phase enters the T-matrix.
2. *Verified conventions.* Every sign, phase, and normalization choice is fixed either by an offline test against an independent reference or by comparison with a full-wave solution of a problem with a known answer (Sec. 6).

2 Theoretical formulation

2.1 Conventions and vector spherical waves

All internal computations use the engineering time convention $e^{+j\omega t}$, consistent with CST. Outgoing waves therefore carry the spherical Hankel function of the second kind, $h_l^{(2)}(kr)$; regular waves carry the spherical Bessel function $j_l(kr)$. Spherical harmonics are orthonormal with the Condon–Shortley phase,

$$Y_{lm}(\theta, \varphi) = \tilde{P}_l^m(\cos \theta) e^{jm\varphi}, \quad \int_{4\pi} Y_{lm} Y_{l'm'}^* d\Omega = \delta_{ll'} \delta_{mm'}, \quad (1)$$

where $\tilde{P}_l^m$ are the fully normalized associated Legendre functions (the normalization of Jackson [7]). With $\gamma_l = 1/\sqrt{l(l+1)}$ and the angular functions

$$\pi_{lm}(\theta) = \frac{m \tilde{P}_l^m(\cos \theta)}{\sin \theta}, \quad \tau_{lm}(\theta) = \frac{d\tilde{P}_l^m(\cos \theta)}{d\theta}, \quad (2)$$

the vector spherical waves are, for radial kind c ($c = 1$: $z_l = j_l$; $c = 3$: $z_l = h_l^{(2)}$),

$$\mathbf{M}_{lm}^{(c)}(\mathbf{r}) = \gamma_l z_l(kr) \left[j\pi_{lm} \hat{\boldsymbol{\theta}} - \tau_{lm} \hat{\boldsymbol{\varphi}} \right] e^{jm\varphi}, \quad (3)$$

$$\mathbf{N}_{lm}^{(c)}(\mathbf{r}) = \gamma_l \left[l(l+1) \frac{z_l(kr)}{kr} \tilde{P}_l^m e^{jm\varphi} \hat{\mathbf{r}} + \frac{[kr z_l(kr)]'}{kr} \left(\tau_{lm} \hat{\boldsymbol{\theta}} + j\pi_{lm} \hat{\boldsymbol{\varphi}} \right) e^{jm\varphi} \right], \quad (4)$$

with $\mathbf{N}^{(c)} = k^{-1} \nabla \times \mathbf{M}^{(c)}$. The tangential vector harmonics obtained from Eqs. (3)–(4) are orthonormal over the sphere. Fields are expanded as

$$\mathbf{E}_{\text{inc}} = \sum_{l=1}^{l_{\max}} \sum_{m=-l}^l a_{lm}^M \mathbf{M}_{lm}^{(1)} + a_{lm}^N \mathbf{N}_{lm}^{(1)}, \quad \mathbf{E}_{\text{sca}} = \sum_{l,m} f_{lm}^M \mathbf{M}_{lm}^{(3)} + f_{lm}^N \mathbf{N}_{lm}^{(3)}, \quad (5)$$

and the magnetic field follows from the curl relations as $Z_0 \mathbf{H} = j \sum a^M \mathbf{N}^{(0)} + a^N \mathbf{M}^{(0)}$ (same radial kind as the corresponding electric term), with Z_0 the free-space impedance. Coefficients are stacked as $[M \text{ block}; N \text{ block}]$ with $l = 1 \dots l_{\max}$ outer and $m = -l \dots l$ inner, giving $N = 2 l_{\max}(l_{\max} + 2)$ modes, and the T-matrix is defined by

$$\mathbf{f} = \mathbf{T} \mathbf{a}. \quad (6)$$

2.2 Plane-wave expansion

A unit-amplitude linear plane wave propagating along $\hat{\mathbf{k}}(\theta_i, \varphi_i)$ with polarization vector $\hat{\mathbf{e}}$ equal to the spherical unit vector $\hat{\boldsymbol{\theta}}$ or $\hat{\boldsymbol{\varphi}}$ at the propagation direction,

$$\mathbf{E}(\mathbf{r}) = \hat{\mathbf{e}} e^{-jk \hat{\mathbf{k}} \cdot \mathbf{r}}, \quad (7)$$

has the regular-wave coefficients

$$a_{lm}^M = 4\pi(-j)^l \hat{\mathbf{e}} \cdot \mathbf{C}_{lm}^*(\hat{\mathbf{k}}), \quad a_{lm}^N = 4\pi(-j)^{l-1} \hat{\mathbf{e}} \cdot \mathbf{B}_{lm}^*(\hat{\mathbf{k}}), \quad (8)$$

with the tangential harmonics $\mathbf{C}_{lm} = \gamma_l(j\pi_{lm} \hat{\boldsymbol{\theta}} - \tau_{lm} \hat{\boldsymbol{\varphi}}) e^{jm\varphi}$ and $\mathbf{B}_{lm} = \gamma_l(\tau_{lm} \hat{\boldsymbol{\theta}} + j\pi_{lm} \hat{\boldsymbol{\varphi}}) e^{jm\varphi}$ evaluated at the propagation direction. Equation (8) was derived by numerical projection onto the basis of Sec. 2.1 rather than transcribed from the literature, and is fixed by a test that reconstructs the plane-wave field from the coefficients to a relative accuracy of 10^{-13} .

2.3 Multipole truncation

Wiscombe’s criterion [9], applied to the size parameter $x = k_{\max}a$ of the circumscribing sphere of radius a ,

$$w(x) = \left\lceil x + 4x^{1/3} + 1 \right\rceil \quad (x \leq 8), \quad w(x) = \left\lceil x + 4.05 x^{1/3} + 2 \right\rceil \quad (8 < x < 4200), \quad (9)$$

is an upper bound rather than a prescription. It gives the order at which the Mie series has converged to machine precision, which is far smaller than the uncertainty of the present extraction (10^{-3} – 10^{-2} in the coefficients; Sec. 7). Multipole orders whose true amplitudes lie below that uncertainty cannot be determined from the data; retaining them adds noise to the matrix and increases the cost twice over, since $l_{\max}$ fixes both the number of modes $N = 2l_{\max}(l_{\max} + 2)$ — and with it the number of solver runs — and the minimum monitor radius (Sec. 2.5), and therefore the size of the simulation domain.

In practice one takes Eq. (9) as the upper bound, determines the per-order amplitudes $|f_l|$ of the scattered field from a single test solve (or from a previous extraction of a similar object), finds the largest order still resolved above the uncertainty, and sets $l_{\max}$ one order higher. Physical and spurious content are easy to tell apart in a stored matrix: physical amplitudes decrease with l and depend on frequency, whereas noise amplified by the ill-conditioning at high orders *increases* with l and is essentially independent of frequency. As an illustration, a gold spoke-and-wheel resonator extracted at the Wiscombe value $l_{\max} = 9$ had 81% of its matrix norm in the $l = 9$ block, constant to four digits from 10 to 34 THz — clearly not a property of the scatterer; the measurable content ended near $l = 4$, and re-extraction at $l_{\max} = 5$ satisfied all diagnostics. The per-frequency Wiscombe order and the fraction of the matrix norm above it are stored with every extraction (`noise_order_content`, Sec. 2.9); a masking utility sets the undetermined orders to zero before they enter sums over modes.

2.4 Self-calibrating field separation

The field monitor records the *total* field $\mathbf{E} = \mathbf{E}_{\text{inc}} + \mathbf{E}_{\text{sca}}$ on a sphere of radius R , chosen for each frequency sub-range by the placement criteria of Sec. 2.5. Projecting the tangential components of $\mathbf{E}$ and $Z_0\mathbf{H}$ onto $\mathbf{C}_{lm}$ and $\mathbf{B}_{lm}$ yields four scalars per mode,

$$p_{lm}^C = \langle \mathbf{C}_{lm}, \mathbf{E}_t \rangle, \quad p_{lm}^B = \langle \mathbf{B}_{lm}, \mathbf{E}_t \rangle, \quad h_{lm}^C = \langle \mathbf{C}_{lm}, Z_0\mathbf{H}_t \rangle, \quad h_{lm}^B = \langle \mathbf{B}_{lm}, Z_0\mathbf{H}_t \rangle, \quad (10)$$

which are related to the unknown incident and scattered coefficients by two 2×2 linear systems per mode,

$$\begin{pmatrix} p_{lm}^C \\ h_{lm}^B \end{pmatrix} = \begin{pmatrix} j_l & h_l^{(2)} \\ j \psi'_l / kr & j \xi'_l / kr \end{pmatrix} \begin{pmatrix} a_{lm}^M \\ f_{lm}^M \end{pmatrix}, \quad \begin{pmatrix} p_{lm}^B \\ h_{lm}^C \end{pmatrix} = \begin{pmatrix} \psi'_l / kr & \xi'_l / kr \\ j j_l & j h_l^{(2)} \end{pmatrix} \begin{pmatrix} a_{lm}^N \\ f_{lm}^N \end{pmatrix}, \quad (11)$$

where $\psi_l = kr j_l(kr)$ and $\xi_l = kr h_l^{(2)}(kr)$ are the Riccati–Bessel functions evaluated at $kr = kR$. The determinants of the two systems in Eq. (11) are proportional to the Wronskian $W\{j_l, h_l^{(2)}\} \neq 0$, so the decomposition of the total field into incident and scattered parts always has a unique solution. Its sensitivity to errors in the field data is another matter: for $kR < l$ the Bessel function $j_l(kR)$ is exponentially small (the classically forbidden region below the centrifugal barrier), the matrix in Eq. (11) becomes nearly singular, and small errors in the recorded field produce large errors in a_{lm} at that order. This is the origin of the first placement criterion in

Sec. 2.5. Because $\mathbf{a}$ and $\mathbf{f}$ are obtained from the same data, any amplitude or phase convention of the source cancels exactly in Eq. (14) below. CST documents its plane-wave source as unit-amplitude with phase reference at the global origin; the extraction confirms this (Sec. 6) but does not rely on it.

Since the incident field is known analytically, comparing the measured coefficients $\mathbf{a}$ with Eq. (8) provides a diagnostic of the fidelity of the recorded field itself — the *incident deviation*, stored per frequency with every extraction. It is nonzero because of the discretization error of the solver and grows when the recorded field is degraded for any reason. The two mechanisms observed in practice are a monitor sphere placed inside the reactive near field of the scatterer’s higher multipoles (Sec. 2.5), which raises the deviation to 10^{-1} , and insufficient mesh resolution at the upper end of the frequency range, which raises it gradually to a few times 10^{-2} . In template mode (Sec. 5.2), where the geometry is supplied by the user rather than generated by the package, a large deviation can also indicate a geometry that is not centered on the origin or a project built in different units than declared; the programmatic path excludes these by construction.

2.5 Monitor placement

The monitor radius R must satisfy two conditions. Both are most restrictive at the lowest frequency of the extraction range, where kR is smallest:

$$(i) \quad k_{\min} R \geq l_{\max} + 2, \quad (12)$$

$$(ii) \quad kR \geq w(ka) + 1 \quad \text{at every frequency of the range,} \quad (13)$$

with w the Wiscombe order of Eq. (9) and a the circumscribing radius. Condition (i) keeps the incident/scattered decomposition of Sec. 2.4 well conditioned at $l = l_{\max}$: for $kR < l$ the matrix elements containing $j_l(kR)$ are exponentially small and the inversion amplifies field errors. Condition (ii) is independent of the truncation. The scatterer radiates multipoles up to order $w(ka)$ whether or not the projection retains them, and orders with $l > kR$ are still evanescent at the monitor radius: their amplitudes grow steeply toward the scatterer, they enlarge the total field recorded on the sphere, and the solver’s relative error on that enlarged field enters every projected coefficient as an absolute error. The left side of (ii) grows with k faster than $w(ka)$ does, so the condition need only be checked at $k_{\min}$.

Both conditions, and the +1 in (ii), come from observed failures. For (i): an extraction with $k_{\min} R = 0.90$ and $l_{\max} = 3$ gave incident deviations up to 0.99, concentrated at $l = l_{\max}$. For (ii): five extractions, all satisfying (i), are summarized by the continuous quantity $n(x) = x + 4x^{1/3} + 1$ (whose integer ceiling is w). Extractions with $k_{\min} R - n = +0.93$, $+0.75$, and $+0.67$ gave incident deviations below 10^{-2} ; those with $+0.44$ and -0.25 gave deviations of 0.11–0.15, singular values of $\mathbf{S}$ up to 1.37, and an $l = l_{\max}$ block roughly an order of magnitude too large, largest at the low-frequency end of the range — and, where two sub-ranges adjoin, discontinuous at their junction, which is what distinguishes this mechanism from insufficient mesh resolution (Sec. 7). The integer ceiling initially obscured the distinction: two of these extractions both had $kR = w = 8$ while differing by 0.31 in $kR - n$, and a rule requiring only $kR \geq w$, consistent with the first four data points, failed on the fifth. Requiring $kR \geq w + 1$ places every future extraction at $kR - n \geq 1$, above all observations that satisfied the diagnostics. `recommended_monitor_factor` computes the smallest R satisfying both conditions and is the default (`monitor_factor=None`); a manually supplied factor is checked at extraction start. Note also the coupling to Sec. 2.3:

condition (i) ties R to $l_{\max}$, so an unnecessarily large truncation enlarges the simulation domain and its memory use roughly as the cube of R — a further reason to choose $l_{\max}$ from the measured multipole content rather than from Eq. (9) alone.

2.6 Least-squares extraction and illumination set

Collecting the coefficient vectors of K illuminations as columns of $N \times K$ matrices $\mathbf{A}$ and $\mathbf{F}$, the T-matrix is the least-squares solution of

$$\mathbf{F} = \mathbf{T} \mathbf{A} \quad \implies \quad \mathbf{T} = \mathbf{F} \mathbf{A}^\dagger \left(\mathbf{A} \mathbf{A}^\dagger \right)^{-1}, \quad (14)$$

computed by a rank-revealing solver. The illumination set consists of $K/2$ directions with two orthogonal polarizations each. Two direction schemes are provided:

Fibonacci (default). Quasi-uniform spiral points, available for any direction count and free of the coordinate poles. The number of directions is $\lceil \eta N/2 \rceil$ with overdetermination factor η (default 1.5); the observed condition number of $\mathbf{A}$ is close to unity.

Lebedev. Nodes of Lebedev quadrature rules [10]. Because the least-squares formulation does not integrate over directions, the quadrature exactness of the rule is not required here; the practical difference from the Fibonacci scheme is that Lebedev rules exist only at specific point counts, so the number of solver runs is rounded up to the next available rule. Rules with 6, 14, and 26 points are built in with exact weights; larger standard rules can be supplied as data files and are verified by integrating spherical harmonics before use.

Overdetermination averages solver noise: in synthetic tests, additive field noise of 10^{-3} produces a T-matrix error of 4×10^{-5} . The computation consists of $K = 2\lceil \eta N/2 \rceil$ solver runs, each of which covers *all* requested frequencies through field monitors placed at exact solver samples (Table 1); symmetry reduction (Sec. 2.7) divides K by up to the group order. The solver treats each frequency sample as a separate linear system, so the total time scales as the *product* of the illumination count and the number of frequency samples, plus a meshing overhead per run (CST regenerates the mesh on every solver start; Appendix A). Reducing either factor helps proportionally; once the illumination count has been reduced by symmetry, the frequency grid is the remaining large factor.

Table 1: Number of frequency-domain solver runs for overdetermination factor $\eta = 1.5$.

$l_{\max}$	modes N	directions	solver runs
2	16	12	24
3	30	23	46
5	70	53	106
8	160	120	240

2.7 Symmetry-reduced illumination

If the scatterer is invariant under a point group G (axial groups C_n and C_{nv} about z are currently supported), the T-matrix commutes with the representation $D(g)$ of every group element on the

mode basis [2]. Each solved illumination $(\mathbf{a}, \mathbf{f})$ therefore yields $|G|$ columns $(D(g)\mathbf{a}, D(g)\mathbf{f})$, of which only one requires a solver run; for C_{4v} ($|G| = 8$) the measured saving is a factor of 7.2–7.8 in solver runs. The reduction is enabled through `ExtractionConfig(symmetry_n_fold, symmetry_mirror_phi0_deg)`. The assumed symmetry should be verified beforehand: geometrically, with the STL-based point-group detector (`postprocess.symmetry.detect_axial_point_group`), and, when a previous extraction of the object exists, by computing $\|\mathbf{T}D - D\mathbf{T}\|/\|\mathbf{T}\|$ for the candidate operations — if this is comparable to the reciprocity deviation of the same matrix, the symmetry holds to within the accuracy of the data, since reciprocity sets the scale of the extraction error.

Three details of the implementation deserve comment, because each involves a sign or ordering convention that a test suite sharing the same convention cannot detect; all three were fixed by comparison with an independently computed quantity.

1. Rotating the illumination direction corresponds to the *inverse* rotation acting on the coefficients: $\mathbf{a}(\theta, \varphi + \alpha) = D(-\alpha)\mathbf{a}(\theta, \varphi)$. The sign of α cannot be checked by internal consistency — tests constructed with either sign are consistent with themselves — so it was fixed by evaluating the plane-wave expansion, Eq. (8), directly at rotated angles and comparing (agreement to 3×10^{-16} for the correct sign; order unity for the wrong one).
2. A vertical mirror plane at azimuth φ_0 acts as $D(\sigma_{\varphi_0})\mathbf{a}(\theta, \varphi, \hat{e}) = s\mathbf{a}(\theta, 2\varphi_0 - \varphi, \hat{e})$ with $s = +1$ for $\hat{\theta}$ -polarization and $s = -1$ for $\hat{\varphi}$ -polarization. The matrix $D(\sigma)$ is obtained numerically, by evaluating each basis mode at the mirrored points, applying the polar-/axial-vector transformation to $(\mathbf{E}, \mathbf{H})$, and projecting back onto the basis; the construction is checked through the involution $D^2 = I$.
3. The solved directions must be spread over the sphere, e.g. by farthest-point selection. The group operations preserve the polar angle θ , so the orbit of a direction adds only azimuthal copies. Taking candidate directions in the order of the Fibonacci spiral, which begins near the pole, concentrates all solved directions in a polar cap; the resulting system of columns had numerical rank 38 of 48 and condition number 4×10^5 , and the matrix obtained from it violated reciprocity and passivity at order unity on measured data even though the same construction reproduced synthetic (noise-free) matrices to 10^{-8} — a reminder that recovery tests without noise say nothing about conditioning. Farthest-point selection of the same number of directions gives condition numbers of 10–25.

Illuminations on the symmetry axis or in a mirror plane have smaller orbits (some group elements map them to themselves) and are excluded automatically. The expansion is applied before the least-squares solve, so matrix elements forbidden by the enforced group vanish identically in the result. Elements forbidden by the full symmetry of the object but not by the enforced subgroup — for a body of revolution under C_{4v} , those with $m - m' = \pm 4$ — are then a direct measurement of the extraction error, since their true values are zero.

This observation extends to a post-processing step (`postprocess.symmetry.project_onto_symmetry`). For a symmetry that has been verified independently, the true T-matrix is invariant under the group, so the non-invariant part of a measured matrix is extraction error; its relative magnitude, $\|\mathbf{T} - P(\mathbf{T})\|_F / \|\mathbf{T}\|_F$ with P the group average, quantifies that error using every matrix element, and $P(\mathbf{T})$ is the corresponding cleaned matrix. The group average is preferable to setting only the forbidden elements to zero: it is the orthogonal projection onto the invariant subspace, so it also averages symmetry-related allowed elements and reduces their error. Two rules govern its

use. The projection may only be applied for a verified symmetry — applied to a symmetry the object lacks, it replaces physics with an assumption. And the raw matrix should be kept on disk with the removed fraction recorded, the projection being applied downstream: the diagnostics of the raw matrix must be examined first, because an error mechanism with a symmetric component (a misplaced monitor, for instance) is only partially removed by P and the projected matrix then appears more accurate than the underlying data are.

2.8 Surface quadrature

The projections in Eq. (11) are evaluated with a product quadrature: Gauss–Legendre nodes in $\cos\theta$ and uniform nodes in φ . This rule integrates spherical harmonics exactly up to a designed degree, like a Lebedev rule, but is available at any order. The design degree accounts for the full angular bandwidth of the recorded field: the incident plane wave carries angular content up to approximately kR on the monitor sphere, so the quadrature is sized for degree $l_{\text{field}} + l_{\text{max}}$ with $l_{\text{field}} = \lceil kR + 4.05(kR)^{1/3} + 2 \rceil$ plus a safety margin. Undersizing this quadrature aliases incident-field content into the extracted modes.

2.9 Diagnostics

Seven quantities are computed per frequency and stored with every T-matrix:

Quantity	Definition	Healthy value
residual	$\ \mathbf{TA} - \mathbf{F}\ _F / \ \mathbf{F}\ _F$	solver accuracy (10^{-3} – 10^{-2})
conditioning	$\text{cond}(\mathbf{A})$	$< 10^2$ (full Fibonacci: ≈ 1.3 ; C_{4v} -reduced: 10 – 1)
passivity	$\max \text{sv}(\mathbf{S})$, $\mathbf{S} = \mathbf{I} + 2\mathbf{T}$	$\leq 1 + \text{noise floor}$ ($\lesssim 1.03$ observed)
unitarity	$\max \text{sv}(\mathbf{S}) - 1 $	≈ 0 (lossless only)
reciprocity	$\ \mathbf{T} - \mathcal{R}(\mathbf{T})\ _F / \ \mathbf{T}\ _F$	residual level near resonance [†]
incident deviation	measured vs. analytic $\mathbf{a}$	$< 10^{-2}$
noise-order content	$\ \mathbf{T}\ _F$ fraction above the per-frequency Wiscombe order $w(ka)$	≈ 0 where $w(ka) \geq l_{\text{max}}$

[†]The extraction error is an *absolute* quantity, set by the field error relative to the incident-field magnitude (Sec. 6.3). Far below resonance $\|\mathbf{T}\|$ is small while this absolute error is unchanged, so the relative quantities (reciprocity, residual) grow toward the low-frequency end of the range even though the underlying data are no worse there. Values at the ends of the range should be read against this expectation. The reciprocity map is

$$\mathcal{R}(\mathbf{T})_{(p',l',m'),(p,l,m)} = (-1)^{m+m'} \mathbf{T}_{(p,l,-m),(p',l',-m')}, \quad (15)$$

which any scatterer composed of reciprocal media must satisfy to within the extraction accuracy, independent of shape. The sign structure of Eq. (15) was determined numerically by requiring consistency with the amplitude reciprocity theorem $\hat{\mathbf{e}}_2 \cdot \mathbf{F}(\hat{\mathbf{k}}_2 \leftarrow \hat{\mathbf{k}}_1; \hat{\mathbf{e}}_1) = \hat{\mathbf{e}}_1 \cdot \mathbf{F}(-\hat{\mathbf{k}}_1 \leftarrow -\hat{\mathbf{k}}_2; \hat{\mathbf{e}}_2)$ [11] evaluated with the package’s own plane-wave and far-field expansions; alternative sign choices fail this test at order unity. A reciprocity violation at order unity therefore indicates an extraction or convention error rather than physics. Passivity and unitarity follow from energy conservation [2].

2.10 Analytic references: homogeneous and coated spheres

For validation, the diagonal T-matrix of a homogeneous sphere is computed from the Mie coefficients a_l, b_l of Bohren and Huffman [8], and that of a core-shell sphere from their coated-sphere generalization. The literature forms are written in the $e^{-i\omega t}$ convention; in the package convention the entries are

$$T_l^{MM} = -b_l^*, \quad T_l^{NN} = -a_l^*, \quad (16)$$

with complex refractive indices conjugated on input (loss corresponds to a negative imaginary part in the $e^{+j\omega t}$ convention). The conjugation in Eq. (16) cannot be detected by internal consistency tests, which are invariant under conjugation of the reference; it was fixed by comparison with the full-wave solution of the $\varepsilon = 4$ sphere (Sec. 6). The coated-sphere implementation is verified in its two exact limits (shell index equal to the background: homogeneous sphere of the core; core index equal to the shell: homogeneous sphere of the outer radius) and by passivity for lossy shells.

2.11 Conversion to the tmat.h5 convention

The data format [12] and *treams* [13] use the physics convention $e^{-i\omega t}$ with outgoing waves $h_l^{(1)}$, Jackson-normalized harmonics, and a basis that differs from Sec. 2.1 by an overall factor i , which cancels in the T-matrix. Writing one physical monochromatic field in both phasor conventions, $\mathbf{E}(\mathbf{r}, t) = \text{Re}[\mathbf{E}_+ e^{+j\omega t}] = \text{Re}[\mathbf{E}_- e^{-i\omega t}]$ with $\mathbf{E}_- = \mathbf{E}_+^*$, and using $\tilde{P}_l^{-m} = (-1)^m \tilde{P}_l^m$, the basis functions conjugate as $(\mathbf{M}_{lm}^{(c)})^* = (-1)^m \mathbf{M}_{l,-m}^{(c')}$ (radial kinds mapping $j_l \rightarrow j_l$, $h_l^{(2)} \rightarrow h_l^{(1)}$), so coefficients map antilinearly, $a_{lm}^- = (-1)^m (a_{l,-m}^+)^*$, and

$$\mathbf{T}_{(p,l,m),(p',l',m')}^- = (-1)^{m+m'} \left[\mathbf{T}_{(p,l,-m),(p',l',-m')}^+ \right]^*. \quad (17)$$

Equation (17) is *not* element-wise conjugation; the two coincide only for matrices that are diagonal and even in m , such as spheres. Applying the map twice returns the original matrix, and the test suite verifies the map at the level of the fields themselves: the complex conjugate of a field evaluated from arbitrary coefficients equals the field evaluated from the mapped coefficients, and transforming a T-matrix and transforming its input/output coefficient vectors give the same scattered field. Files on disk are written in the converted convention with explicit mode tables; the reader applies the inverse map, so quantities analyzed inside the package and quantities read from disk agree.

3 Simulation setup in CST Studio Suite

The extraction imposes the following configuration, generated as history blocks through the CST scripting interface.

Solver. The frequency-domain solver (FDFD) on a tetrahedral mesh with second-order elements. (CST’s time-domain solver, an FDTD-type method, is not used here.) The active solver must be switched explicitly (`ChangeSolverType`); a new project defaults to the time-domain solver, and configuring the frequency-domain solver alone does not change the active solver.

Mesh strategy. *Deterministic, non-adaptive* by default: a fixed steps-per-wavelength density (`ScattererPlan.mesh_steps_per_wave`, with the `CellsPerWavelengthPolicy` pinned — see Appendix A) and `mesh_adaption=False`. CST re-runs adaptive refinement from scratch on *every* solver start and never carries an adapted mesh across solves (Appendix A), so with adaptation enabled each illumination pays the full multi-pass cost and each column of the T-matrix is measured on a different mesh. A fixed density makes every solve identical and was validated on this pipeline against both an adaptive reference and the exact Mie solution at matched accuracy and roughly a quarter of the solve time. The density must be *calibrated per geometry class* by a short convergence check (compare the scattered column across densities and against one adaptive run; `examples/density_check_spoke_wheel.py` is the template).

Splitting the frequency range. Over a wide spectral range the monitor radius is fixed by the lowest frequency (Sec. 2.5) while the mesh density is fixed by the highest, and a single project must accommodate both: a large domain meshed finely. For an extraction spanning 10–34 THz this combination produced a mesh of order 10^7 cells, and the solver ran out of memory while constructing the preconditioner. The remedy is to divide the frequency range into sub-ranges, each with its own monitor radius and mesh (`range_edges_thz` in the configuration; `BAND_EDGES_THZ` in the extraction scripts); the same 10–34 THz extraction split at 20 THz required well under 10^6 cells per sub-range. Each sub-range is an independent extraction writing its own file; the files are merged afterwards by concatenation along the frequency axis (`examples/merge_band_tmatrices.py`, which verifies identical mode bases and non-overlapping frequency grids). After merging, the continuity of $T(f)$ at the junction of two sub-ranges is evaluated automatically and reported as a diagnostic: the two sides of the junction were computed with different meshes and different monitor radii, so a step in $T(f)$ there that is no larger than the steps between neighboring frequencies within a sub-range shows that the result is insensitive to both choices.

Excitation. Plane-wave source with linear polarization; the propagation direction and polarization vector are set per illumination. CST normalizes the excitation to the specified electric field vector (V/m) with phase reference at the global origin.

Boundaries and background. Open boundaries with added space set *explicitly on all six faces*, and the background material set explicitly to vacuum. Both are set explicitly rather than left to defaults so that the configuration is complete and reproducible from the history alone. Note that in scripted use the “apply in all directions” option does not copy a single face’s added space to the other faces (Appendix A); the resulting asymmetric domain affected all early extractions until each face was set individually. No symmetry planes are used: the illumination set spans arbitrary directions, and symmetry is exploited algebraically instead (Sec. 2.7).

Monitors. One electric-field and one magnetic-field monitor per extraction frequency, with the monitor frequencies forced as exact solver samples (never interpolated). For plane-wave excitation the results appear in the navigation tree with the label suffix `[pw]`.

Field export. Complex E and H are exported at the surface quadrature nodes through the `ASCIIExport` object with a user-supplied evaluation-point file. Exported coordinates are compared against the requested points, with automatic detection of a global unit scale; a mismatch aborts the run. Magnetic fields are exported in A/m and multiplied by Z_0 .

Result lifetime and caching. Redefining the plane wave invalidates solver results, so the fields of each illumination are exported immediately after its solve. Completed illuminations are written to disk (`.npz`), and an interrupted extraction resumes from them without recomputation. Each cached record stores the frequency grid, the illumination direction and polarization, and the monitor radius, and is reused only when all of these match the current run. This matters because the illumination list and the monitor radius can legitimately differ between two runs of the same object — symmetry reduction changes the list of solved directions, and a revised monitor radius changes the surface the fields were recorded on — so a cache matched by position in the list alone could supply a column belonging to a different direction or a different monitor.

Run log. Everything printed during an extraction (including warnings and the diagnostics table) is appended to `exports/log.txt` in the run directory with a timestamped header per invocation.

4 Software

4.1 Modules

Module	Content
<code>vswf.py</code>	basis functions, projections, plane-wave expansion, separation, convention maps
<code>quadrature.py</code>	surface quadrature; Fibonacci and Lebedev direction sets
<code>mie.py</code>	homogeneous and coated sphere references; Wiscombe criterion
<code>tmatrix_solve.py</code>	least-squares solve; passivity, unitarity, reciprocity
<code>storage.py</code>	<code>tmat.h5</code> writing and reading with convention conversion
<code>pipeline.py</code>	orchestration: project setup, illumination loop, caching, monitor-placement criteria, noise-order diagnostics
<code>config.py</code>	user settings (directories), extraction options, automatic-check thresholds
<code>cst_driver/</code>	CST session control, geometry builders, template mode, excitation, monitors, solver configuration, field export
<code>postprocess/symmetry.py</code>	point-group representations, symmetry-reduced illumination, STL point-group detector, group-average denoising
<code>postprocess/metasurface.py</code>	forward-scattering amplitude and specular S-parameters (isolated-scatterer approximation)
<code>postprocess/coupling.py</code>	finite-cluster Foldy–Lax: translation operators, cluster solve, separation guards

The package is self-contained: it depends only on `numpy`, `scipy`, `h5py`, and the CST Python interface.

4.2 Installation and configuration

Create a Python environment (≥ 3.9) with `numpy`, `scipy`, `h5py`, and `pytest`, and install the CST interface shipped with the CST installation:

```
pip install --no-index --find-links \
    "<CST folder>/Library/Python/repo/simple" cst-studio-suite-link
```

Machine-specific settings are collected in an optional JSON file `settings.json` at the repository root, written once per machine:

```
{
  "run_root": "D:/cst_runs",
  "library": "",
  "max_cpus": 6,
  "hardware_acceleration": false
}
```

`run_root` (CST projects and solver output) must be a local disk, because cloud-synchronized folders lock files during solves; `library` (destination of the final `.tmat.h5` files) defaults to `library/` in the repository. The CPU count and the hardware-acceleration setting belong here because they differ between machines while the physics configuration does not; an extraction configuration can override them for a single run. The environment variables `CST_TMATRIX_RUN_ROOT` and `CST_TMATRIX_LIBRARY` take precedence over `settings.json` where both are set.

4.3 Operating procedure

1. **Offline verification** (no CST required): `python -m pytest tests/ -q`. All tests must pass before any extraction.
2. **Single-solve verification** (requires CST): `python examples/smoke_test_sphere.py` builds a dielectric sphere, runs one solve, and checks the measured incident wave against the analytic plane-wave expansion and the measured scattered wave against Mie theory. Run this once per machine or CST version.
3. **Extraction.** Describe the scatterer by a `ScattererPlan` and call `extract_tmatrix` (see Sec. 5).

4.4 Automatic checks and unattended operation

An extraction takes hours, but most of the conditions that invalidate one can be tested in seconds to minutes. `extract_tmatrix` therefore performs three checks automatically (thresholds in `ExtractionConfig`; a failed check raises `ExtractionCheckError` and stops the run):

Before any computation. The two monitor-placement conditions of Sec. 2.5 are required, not merely reported. The automatically computed monitor factor satisfies them by construction; the check protects against configurations that bypass the automatic factor — an explicit `monitor_factor` in a script, an old configuration file, or a future code change — reaching the solver.

After meshing, before any solve. The mesh cell count and the estimated solver memory (≈ 12 kB per second-order tetrahedron; Appendix A) are compared with configured limits and with the memory currently available. A configuration that would exhaust memory is rejected after the few minutes of meshing rather than discovered when the solver fails.

After every solve. The incident deviation of the completed solve is evaluated immediately.

Above 10^{-2} (default) it is logged; above 3×10^{-2} (default) the extraction stops, since a recorded field this far from the known incident wave invalidates the column regardless of the cause — including causes not yet understood. Columns from a stopped solve are not written to the cache, so a corrected rerun cannot reuse them unnoticed. After the first solve the measured top-order content (Sec. 2.3) and an estimate of the total run time are also printed, the latter with an optional upper limit.

As a point of reference: the per-solve check detected the misplaced monitor of Sec. 2.5 after 21 minutes; the same physical mistake, made before these checks existed, was found only after a completed 5.5-hour run.

For extracting several designs without supervision, `examples/batch_extract_spoke_wheel.py` runs the designs sequentially, verifies the placement conditions for every sub-range before CST is started, opens a fresh CST connection per sub-range (so a crash of the CST front end in one does not affect the others), continues with the remaining work if one fails, merges each design’s sub-ranges, evaluates the continuity of $T(f)$ at their junctions, and appends a summary to `batch_log.txt`. A sub-range stopped by the incident-deviation check is rerun once automatically with the monitor radius increased by one Wiscombe order; a second stop is left for the operator.

Solver resources (thread count, hardware acceleration) are properties of the machine, not of the physics problem, and are therefore read from the per-machine `settings.json` (Sec. 4.2) rather than fixed in the extraction configuration; a configuration may still override them for a single run.

5 Usage

5.1 Scripted geometries

```
import numpy as np
from cst_tmatrix import ScattererPlan, ExtractionConfig, extract_tmatrix
from cst_tmatrix.cst_driver import CSTSession, builder

def build(h):
    builder.define_material(h, "si", eps=11.7)
    builder.new_component(h, "scatterer")
    builder.build_cylinder(h, radius=30.0, zmin=-25.0, zmax=25.0,
                           material="si")      # project length units

plan = ScattererPlan(name="si_disk", build=build, r_circ_m=39.1e-6,
                     freqs_hz=np.array([0.6e12, 0.8e12, 1.0e12]))
T, diagnostics, path = extract_tmatrix(CSTSession(), plan)
```

Arbitrary geometry is supported through `h.add_history(caption, vba)` with any valid CST history block. The geometry must be centered at the origin, which is the origin of the multipole expansion.

5.2 Template projects

Complex geometries are often easier to build interactively. In template mode the user supplies a finished CST project containing *only* the geometry and materials:

```
plan = ScattererPlan(name="my_object", template=r"path\to\object.cst",
                     r_circ_m=50e-6, freqs_hz=np.array([1.0e12]),
                     length_unit="um", freq_unit="THz")
```

The file is copied into the run directory (the original is never modified), verified, and completed with the extraction settings of Sec. 3. Requirements on the template:

- geometry centered at the origin;
- no ports and no periodic or unit-cell boundaries;
- no structure extending to the domain boundary (in particular no substrate slab): the monitor sphere must lie in homogeneous background;
- built in the declared length and frequency units (units are not modified, because reinterpreting units would rescale the geometry).

The verification reports PASS/FAIL/UNVERIFIED per check; the bounding box of all solids is compared against the declared circumscribing radius, which catches decentered geometry and unit or radius mistakes. Checks that the scripting interface cannot perform are reported as UNVERIFIED for manual confirmation.

5.3 Extraction from a configuration file

An extraction can be specified entirely by a JSON configuration file; no Python is written. The geometry is built interactively in CST (template requirements, Sec. 5.2), the file is filled in following the annotated example `examples/example_extraction.json`, and the extraction is run with

```
python -m cst_tmatrix.runner my_extraction.json --dry-run
python -m cst_tmatrix.runner my_extraction.json
```

With `--dry-run` the configuration is validated and the monitor-placement quantities are printed for every frequency sub-range; CST is not started. Without it, the sub-ranges are extracted in sequence with the defaults of this manual (automatic monitor placement, fixed mesh density, the automatic checks, symmetry reduction if configured), merged, tested for continuity at the sub-range junctions, and written as one `.tmat.h5` file.

The configuration file contains the quantities that describe the problem: the geometry file, the circumscribing radius, the frequency grid and its subdivision, $l_{\max}$, and optionally the point-group symmetry of the cell. Machine-dependent settings (working directory, CPU count, hardware acceleration) are kept separately in `settings.json`, written once per machine (Sec. 4.2). Since JSON has no comment syntax, keys beginning with an underscore are ignored and may hold notes; the example file uses them for its documentation. An invalid or incomplete configuration is reported at load time with a message naming the offending entry.

5.4 Extraction options

`ExtractionConfig` sets the truncation order (`lmax`), the overdetermination factor (`illumination_factor`), the distribution of incidence directions (`illumination_scheme = "fibonacci" or "lebedev"`), the surface-quadrature oversampling, the symmetry reduction (Sec. 2.7), and the thresholds of the automatic checks: the mesh-size and memory limits, the two incident-deviation levels, and an optional limit on the projected run time. The monitor factor is computed automatically by default (Sec. 2.5); the mesh is the fixed density of Sec. 3; a wide frequency range should be subdivided as described there.

Before the first long extraction of a new class of geometry, two short preparatory computations are worthwhile. A mesh-density comparison (solve one illumination at two or three densities and against one adaptive run, and compare the scattered coefficients; `examples/density_check_spoke_wheel.py` shows the procedure) establishes the density the geometry requires. A timing run of two illuminations at the full configuration then gives the incident deviation and the time per solver run, from which the duration of the complete extraction follows by multiplication.

5.5 Output files

Each extraction writes one HDF5 file in the `tmat.h5` layout:

Element	Content
<code>/tmatrix</code>	(n_f, N, N) complex, convention of Sec. 2.11
<code>/frequency</code>	frequencies with <code>unit</code> attribute (Hz)
<code>/modes/l, m, polarization</code>	mode tables; "electric"/"magnetic" parity labels
<code>/embedding</code>	background medium (vacuum)
<code>/scatterer</code>	geometry and material metadata
<code>/computation</code>	method, parameters, and <code>analysis/</code> diagnostics
root attributes	name, description, keywords, format version

The mode tables make the file self-describing; the reader (`storage.load_tmatrix`) accepts any parity-basis ordering and reconstructs the package-internal representation.

6 Validation

6.1 Offline tests

The suite (92 tests, no CST required) verifies: normalized Legendre and angular functions against `scipy`; ladder identities against finite differences; projection round trips at 10^{-13} ; the plane-wave expansion at 10^{-13} ; synthetic T-matrix recovery from fields with randomized source phases at 4×10^{-15} ; noise averaging (10^{-3} field noise $\rightarrow 4 \times 10^{-5}$ matrix error); the far-field projection route; the reciprocity map against the amplitude reciprocity theorem at 10^{-12} ; the coated-sphere reference in both exact limits; passivity and unitarity of lossy and lossless references; Lebedev rule exactness; the convention conversion at the field level; storage round trips through HDF5; the monitor-placement criteria against the recorded placement observations (Sec. 2.5); the point-group representations, orbit conventions, orbit degeneracy, and conditioning of the reduced illumination system (Sec. 2.7), including full-rank recovery of a random C_{4v} -symmetric matrix from the reduced set at 10^{-8} ; the noise-order diagnostics; and

the finite-cluster Foldy–Lax machinery (translation-operator field reproduction, ρ -independence, Born limit, and separation guards, Sec. 8.1).

One limitation of synthetic testing should be stated explicitly: noise-free recovery tests do not probe conditioning. The clustered illumination directions of Sec. 2.7 reproduced synthetic T-matrices to 10^{-8} , yet the same construction failed on measured data with a field error of only 10^{-3} , because the column system had condition number 4×10^5 . The suite therefore asserts condition numbers in addition to recovery errors.

6.2 Comparison with full-wave reference solutions

A dielectric sphere ($\varepsilon = 4$, radius 40 μm , $ka = 0.63$ – 1.26) was extracted with CST Studio Suite 2026 at 0.5, 0.75, and 1.0 THz using $l_{\text{max}} = 3$ (46 solver runs). The comparison with the analytic Mie T-matrix gives:

f (THz)	residual	cond(A)	maxsv(S)	incident dev.	max. rel. error vs. Mie
0.50	1.3×10^{-3}	1.3	1.000066	3.8×10^{-2}	3.4×10^{-3}
0.75	1.0×10^{-3}	1.3	1.000174	1.4×10^{-2}	3.3×10^{-3}
1.00	9.2×10^{-4}	1.3	1.000385	9.8×10^{-3}	3.0×10^{-3}

Off-diagonal elements, which vanish analytically for a sphere, are recovered at 3–4 orders of magnitude below the dominant elements; they measure the extraction noise floor, since the least-squares solve imposes no symmetry. The single-solve verification recovers the incident wave with amplitude 1.0013 and phase 6×10^{-4} rad, confirming CST’s documented source normalization, and matches the Mie prediction of the scattered coefficients to 1.6×10^{-3} .

This comparison exposed two convention errors that offline testing could not: the missing conjugation of the Mie reference, Eq. (16), and the necessity of switching the active solver explicitly (Sec. 3). Both are corrected and documented; the general lesson is that internal consistency tests are blind to any error that amounts to a conjugation of the reference, and at least one comparison against an independent full-wave solution is required to fix the convention.

6.3 The absolute scale of the extraction error

The error of the extracted coefficients is set by an absolute scale, not a relative one. The recorded total field is dominated by the incident wave, and the solver’s relative error $\epsilon \approx 10^{-3}$ on that field enters the projected incident and scattered coefficients with the same *absolute* magnitude, $\sim \epsilon \|\mathbf{a}\|$. This was verified on a weakly scattering sphere ($|T| \approx 0.025$): at every multipole order, the absolute error of the measured incident coefficients (whose exact values are known) and of the scattered coefficients agreed within a factor of two.

Two consequences follow for the interpretation of comparisons. First, for a weak scatterer the ratio of incident to scattered amplitude is large (255:1 in the case above), so the *relative* error of the scattered coefficients is large even when the extraction is working correctly: an apparent 19% disagreement with the exact Mie solution consisted of dominant coefficients agreeing to 0.5–3% together with coefficients whose exact values lie below the absolute error scale and which therefore contain only noise. Second, the same effect appears at the off-resonant ends of a resonator’s frequency range, where $\|\mathbf{T}\|$ is small (Sec. 2.9). A single relative norm over an oversized mode space is therefore not a meaningful error measure; comparisons should be made order by order against the measured absolute scale.

6.4 Benchmark against an independent solver: TiO₂ cylinder

The end-to-end pipeline was validated against the *treams* example T-matrix of a TiO₂ cylinder (radius 250 nm, height 300 nm, $n = 2.5$, vacuum), computed independently with JCMSuite (rotationally-symmetric 2D FEM with built-in multipole expansion) — a different solver, mesh type, and extraction method, at a strong scatterer ($\|T\| \approx 2.5$ – 2.9), for which the absolute error scale of Sec. 6.3 is far below the signal. Extraction: the CST frequency-domain solver on a tetrahedral mesh at $l_{\max} = 4$ (matching the reference), C_{4v} symmetry reduction (10 solver runs instead of 72), deterministic non-adaptive mesh, at 320 THz (off-resonance) and 393.6 THz (the strongest extinction resonance in the range). Two independent comparisons: comparison:

1. *Column-wise, no fit involved*: every solved illumination satisfies $\mathbf{f}_{\text{meas}} = \mathbf{T}_{\text{ref}} \mathbf{a}_{\text{meas}}$ to 0.4–0.8% — a direct test of the CST solve, field export, projection, and separation chain against independent physics, with no free parameters.
2. *Matrix-wise*: relative Frobenius difference 0.73% (320 THz) and 1.13% (393.6 THz); the six dominant entries agree to 0.1–0.2% individually; the extinction observable to 0.15–0.5%. Two structural properties *not* imposed by the extraction emerged from the data: m -conservation (the C_{4v} expansion only enforces $m - m' \equiv 0 \pmod{4}$; the continuous rotational symmetry appeared at the 0.5–0.8% level) and reciprocity (0.9–1.4%). Entries that are structural zeros in the 2D reference are recovered at a median of 2×10^{-16} (C_{4v} -forbidden, enforced) or below 8.5×10^{-3} (allowed-but-unpopulated, the measured noise floor).

6.5 Recommended validation sequence for new object classes

Each step adds one physical feature with an available check: a lossy sphere (exact Mie; passivity strictly below unity); a coated sphere (the last object with an exact reference); a finite cylinder — *done*, Sec. 6.4, against an independent full-wave reference rather than only a selection rule; a sphere dimer (comparison with the translation-addition composition of the single-sphere T-matrix, now available offline in `postprocess/coupling.py`); a cube (C_{4v} selection rules; mesh convergence at edges); a helix (reciprocity nontrivial; magneto-electric coupling blocks). For extractions performed in sub-ranges, the junction continuity test of Sec. 3 applies to every new design at no cost.

7 Sources of error and how they manifest

Each mechanism below has been observed on this pipeline. Several of them produce superficially similar diagnostics; the frequency dependence and the multipole distribution of the anomaly usually identify the cause. The diagnostics of a stored matrix should be read against this list before the matrix is used.

Monitor inside the reactive near field (condition ii, Sec. 2.5). Incident deviation of order 10^{-1} , roughly constant across the frequency range and largest at the *low*-frequency end; singular values of $\mathbf{S}$ well above 1; the $l = l_{\max}$ block of $\mathbf{T}$ an order of magnitude too large; and, in a merged file, a jump of an order of magnitude in reciprocity at the junction of two sub-ranges. The junction discontinuity and the low-frequency weighting distinguish this from insufficient mesh resolution.

Ill-conditioned decomposition at $l = l_{\max}$ (condition i). Incident deviation up to order unity, concentrated at $l = l_{\max}$ and worst at the lowest frequency; occurs when $k_{\min}R$ is at or below $l_{\max}$.

Undetermined multipole orders. Per-order content that *grows* with l and is nearly frequency-independent. The physical orders are unaffected — each (l, m) pair is determined by an independent 2×2 system — but norms and sums over all modes are contaminated. Occurs when $l_{\max}$ is set well above the measurable content (Sec. 2.3).

Small- $\|T\|$ inflation of relative quantities. Residual and reciprocity growing toward the off-resonant ends of the range while the incident deviation does not; a consequence of the absolute error scale, not a defect (Sec. 6.3).

Insufficient mesh resolution at the upper end of the range. All diagnostics degrading smoothly and monotonically toward the highest frequency, most strongly for the electrically largest object of a family; observed up to incident deviation 1.6×10^{-2} and $\max_{\text{sv}}(\mathbf{S}) = 1.05$ at an upper range limit $2.7\times$ above the design resonance. The direction of the trend separates this from the misplaced monitor: mesh resolution is worst at the top of the range and varies smoothly; the misplaced monitor is worst at the bottom, nearly flat, an order of magnitude larger, and discontinuous at a sub-range junction. If the upper end of the range matters for the application, raise the mesh density for that sub-range; otherwise the stored diagnostics record the magnitude of the effect.

Ill-conditioned reduced illumination set. Plane-wave observables (e.g. extinction) correct while the matrix violates reciprocity, passivity, and the object’s known selection rules; condition number of the expanded column system $\gg 10^2$ with numerical rank below N . Occurs when the solved directions cluster on the sphere (Sec. 2.7).

Insufficient memory (“Could not compute preconditioner”). Meshing succeeds and the first frequency sample fails. Produced by the combination of a large monitor radius (set by the lowest frequency) with a fine mesh (set by the highest) over a wide spectral range in one project. Subdivide the range (Sec. 3); the estimate of ≈ 12 kB per second-order tetrahedron against available memory predicts the problem before solving.

Columns computed on different meshes. Occurs if adaptive refinement is left on (each solve adapts independently) or mesh settings change mid-run. There is no single sharp symptom, which is why the mesh must be fixed before the illumination loop begins (Sec. 3).

Reuse of cached columns from a different configuration. Excluded by construction: cached columns are reused only when the frequency grid, illumination direction, polarization, and monitor radius all match (Sec. 3).

8 Planned extensions

8.1 Metasurface post-processing: implemented stages

Post-processing consumes stored T-matrices; the extraction machinery is not involved. Two stages are implemented and validated offline:

Specular S-parameters in the isolated-scatterer approximation (`postprocess/metasurface.py`). With the package scattering amplitude $\mathbf{F}$ defined by $\mathbf{E}_{\text{sca}} \rightarrow \mathbf{F}(\theta, \varphi) e^{-jkr}/(kr)$, the forward-scattering theorem gives, for a periodic array of cell area A under incidence $(\mathbf{k}_{\text{inc}}, \hat{\mathbf{e}}_{\text{inc}})$,

$$S_{21} = 1 - \frac{j 2\pi}{k^2 A \cos \theta_{\text{inc}}} \hat{\mathbf{e}}_{\text{inc}}^* \cdot \mathbf{F}(\mathbf{k}_{\text{inc}}), \quad S_{11} = -\frac{j 2\pi}{k^2 A \cos \theta_{\text{inc}}} \hat{\mathbf{e}}_{\text{ref}}^* \cdot \mathbf{F}(\mathbf{k}_{\text{ref}}). \quad (18)$$

Both the $1/k^2$ (from the dimensionless normalization of $\mathbf{F}$ above) and the sign of j (from the $e^{+j\omega t}$ convention) differ from the textbook $e^{-i\omega t}$ form; each was initially transcribed wrongly and fixed by an independent check — the magnitude against a direct CST periodic simulation, the sign by the optical theorem $\text{Im}[\hat{\mathbf{e}}^* \cdot \mathbf{F}_{\text{fwd}}]/k = -(k/4\pi) \sigma_{\text{ext}}$ evaluated on an exact Mie sphere. Transcribing phasor-convention-sensitive formulas without such a check is not safe in this codebase. The isolated-scatterer approximation itself carries a real physical error (inter-cell coupling is absent); comparison against a direct periodic CST solve of a resonant cell shows qualitative agreement with quantitative gaps that the Foldy–Lax stage is designed to close.

Finite-cluster Foldy–Lax (`postprocess/coupling.py`). Self-consistent outgoing coefficients $\mathbf{b}_i = \mathbf{T}_i(\mathbf{a}_i + \sum_{j \neq i} \mathbf{C}(\mathbf{r}_i - \mathbf{r}_j) \mathbf{b}_j)$ for a finite cluster [4, 5]. The translation-addition operators $\mathbf{C}$ are constructed *numerically* from the package’s own validated field primitives (evaluate an outgoing mode about one centre, re-project onto regular modes about the other) rather than from closed-form Gaunt/Wigner coefficients — the same convention-risk argument as above. Two measured facts govern accuracy: the re-projection sphere must have the radius of the *receiving scatterer*, never a fraction of the separation (a regular expansion on a sphere of radius ρ carries angular content $\sim k\rho$, so ρ tied to the separation destroys the truncated expansion — measured error 0.66 at $k\rho = 20$ with $l_{\text{max}} = 8$ versus 10^{-5} – 10^{-6} with $\rho = a$); and accuracy degrades as circumscribing spheres approach ($\sim 10^{-3}$ at twice the touching distance, $\sim 6 \times 10^{-2}$ at $1.3\times$), guarded by `check_pair_separation`.

8.2 Planned: periodic lattice sum and substrate coupling

The infinite-array specular response requires the lattice sum $\Omega = \sum_{\mathbf{R} \neq 0} \mathbf{C}(\mathbf{R}) e^{j\mathbf{k}_{\parallel} \cdot \mathbf{R}}$, which is conditionally convergent and requires Ewald summation; the per-site algebra $\mathbf{T}_{\text{eff}} = (\mathbf{I} - \mathbf{T} \Omega)^{-1} \mathbf{T}$ is already implemented and tested against its defining equation. For a scatterer above a substrate, the free-space $\mathbf{T}$ -matrix is coupled to the interface through a reflection operator $\mathbf{R}$ constructed by expanding outgoing spherical waves into plane waves (Weyl identity), applying Fresnel reflection, and projecting back: $\mathbf{T}_{\text{coupled}} = (\mathbf{I} - \mathbf{T} \mathbf{R})^{-1} \mathbf{T}$. The acceptance test for the combined stage is the comparison of the composed unit-cell response against a direct CST simulation of the same cell with periodic (unit-cell) boundaries; CST’s multilayered background for plane-wave excitation offers an independent check of the substrate operator alone.

A CST-specific behaviors and workarounds (bug report)

Every item below was established by controlled experiment against CST Studio Suite 2026 on this pipeline (dates given); none is documented in the CST help in the form stated here. They are collected in one place because each silently corrupted or blocked real extractions before it was found.

1. **Background.ApplyInAllDirections does not broadcast in scripted history blocks** (2026-08-05). Setting one face distance (e.g. `.XminSpace`) with `.ApplyInAllDirections`

"True" pads *only that face*; the flag is a GUI convenience. Verified via `Boundary.GetCalculationBox()`: the domain was asymmetric (zero-padded in y and z) in every extraction until all six `.XminSpace...ZmaxSpace` were emitted explicitly. Workaround: `builder.set_background_vacuum` writes all six.

2. **MeshSettings steps-per-wavelength values are silently ignored under the default policy** (2026-08-05). `CellsPerWavelengthPolicy` defaults to "automatic", in which CST computes its own density and disregards `StepsPerWaveNear/Far` entirely: settings of 15/6, 30/12, and 50/20 all produced the identical cell count until the policy was pinned to "cellspervavelength", after which the same three settings gave 82k, 599k, and 2.33M cells. Any workflow that believes it is controlling tetrahedral density without pinning this policy is not.
3. **No adaptive-mesh reuse across solver runs** (2026-08-05). `FDSolver.MeshAdaptionTet` is re-evaluated on every `.Start`; an adapted mesh is never carried to the next solve. Three mechanisms were tested and refuted in continuous sessions with the same project handle: (a) toggling the flag off after a converged adaptive solve *reverts to the coarse base mesh* (1,022,285 $\rightarrow$ 81,734 cells) rather than freezing the adapted one; (b) changing the excitation via `StoreDoubleParameter+RebuildOnParametricChange` (the result-preserving parametric path) behaves identically; (c) driving the sweep through CST's own `ParameterSweep` engine costs the full per-point adaptation ($2\times$ single-solve time for 2 points). Consequence: adaptation, if enabled, is paid per illumination on a per-column mesh; the pipeline's default is the deterministic fixed density of Sec. 3 instead.
4. **MeshAdaption3D.MinPasses rejects 1** (2026-08-05). Values below 2 raise "Please specify an integer number greater or equal 2."
5. **MaxCPUs requires LimitCPUs "True"** (2026-08-03). Without the (undocumented) `LimitCPUs` flag, the thread count in `MaxCPUs` is ignored and all cores are used. GPU offload uses the single-argument history form `.HardwareAcceleration "True"`.
6. **Killing the controlling Python process does not abort an in-flight solve** (2026-08-06). The solver (`Solver_HF_Tet_FD_AMD64`) is a separate process that continues to completion, holding CPU cores and memory; two live incidents contaminated subsequent timing runs. After stopping a run mid-solve, check for and terminate the solver process explicitly.
7. **"Could not compute preconditioner" is the out-of-memory signature** (2026-08-06). Meshing succeeds, the first frequency sample dies. Observed calibration: ~ 12 kB per second-order tetrahedron; a $\sim 10^7$ -cell configuration exceeds a 32 GB machine. See the subdivision of the frequency range, Sec. 3.
8. **PlaneWave.Normal/.EVector accept project-parameter names despite double-typed signatures** (2026-08-05). The general rule that VBA double parameters reject parametric expressions does not hold here; `GetNormal()` confirms the values track `StoreDoubleParameter` updates after `RebuildOnParametricChange`. (Exploited only diagnostically; the pipeline redefines the plane wave per illumination.)
9. **Mesh.Update() silently returns zero cells without the high-frequency solver context** (2026-08-06). Callable only after `ChangeSolverType` and solver configuration;

`ForceUpdate()` alone invalidates without rebuilding. Also, `Mesh.Update` does *not* pick up changed `MeshSettings` densities in an already-meshed session (item 2 interacts); a fresh project reflects them.

10. **Operational: Python stdout block-buffering hides live progress** (2026-08-05). When the interpreter’s output is piped (logs, IDE consoles), prints buffer for minutes-to-hours; run extraction scripts with `python -u`.

References

- [1] P. C. Waterman, “Matrix formulation of electromagnetic scattering,” *Proc. IEEE* **53**, 805–812 (1965).
- [2] P. C. Waterman, “Symmetry, unitarity, and geometry in electromagnetic scattering,” *Phys. Rev. D* **3**, 825 (1971).
- [3] M. I. Mishchenko, L. D. Travis, and A. A. Lacis, *Scattering, Absorption, and Emission of Light by Small Particles* (Cambridge University Press, 2002).
- [4] L. L. Foldy, “The multiple scattering of waves,” *Phys. Rev.* **67**, 107–119 (1945).
- [5] L. Tsang, J. A. Kong, and K.-H. Ding, *Scattering of Electromagnetic Waves* (Wiley, 2001).
- [6] M. Fruhnert, I. Fernandez-Corbaton, V. Yannopapas, and C. Rockstuhl, “Computing the T-matrix of a scattering object with multiple plane wave illuminations,” *Beilstein J. Nanotechnol.* **8**, 614–626 (2017).
- [7] J. D. Jackson, *Classical Electrodynamics*, 3rd ed. (Wiley, 1999).
- [8] C. F. Bohren and D. R. Huffman, *Absorption and Scattering of Light by Small Particles* (Wiley, 1983).
- [9] W. J. Wiscombe, “Improved Mie scattering algorithms,” *Appl. Opt.* **19**, 1505–1509 (1980).
- [10] V. I. Lebedev, “Quadratures on a sphere,” *USSR Comput. Math. Math. Phys.* **16**, 10–24 (1976).
- [11] D. S. Saxon, “Tensor scattering matrix for the electromagnetic field,” *Phys. Rev.* **100**, 1771 (1955).
- [12] N. Asadova *et al.*, “T-matrix representation of optical scattering response: Suggestion for a data format,” *J. Quant. Spectrosc. Radiat. Transf.* **333**, 109310 (2025).
- [13] D. Beutel, I. Fernandez-Corbaton, and C. Rockstuhl, “treams – a T-matrix-based scattering code for nanophotonics,” *Comput. Phys. Commun.* **297**, 109076 (2024).
- [14] W. J. Padilla, *The Physics of the Neural Analytic Modal Method* (internal document).